

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: data = pd.read_csv("language.csv")
```

```
In [3]: data
```

```
Out[3]:
```

	Text	language
0	klement gottwaldi surnukeha palsameeriti ning ...	Estonian
1	sebes joseph pereira thomas på eng the jesuit...	Swedish
2	ถนนเจริญกรุง อักษรโรมัน thanon charoen krung ...	Thai
3	விசாகப்பட்டினம் தமிழ்ச்சங்கத்தை இந்துப் பத்திர...	Tamil
4	de spons behoort tot het geslacht haliclona en...	Dutch
...	...	...
21995	hors du terrain les années et sont des année...	French
21996	ใน พศ. หลังจากที่ได้เสร็จประพาสแหลมมลายู ชาว จีน...	Thai
21997	con motivo de la celebración del septuagésimoq...	Spanish
21998	年月, 當時還只有歲的她在美國出道, 以mai-k名義推出首張英文《baby i like》, 由...	Chinese
21999	aprilie sonda spațială messenger a nasa și-a ...	Romanian

22000 rows × 2 columns

```
In [7]: data.isnull().sum()
```

```
Out[7]: Text      0
language    0
dtype: int64
```

```
In [4]: from sklearn.feature_extraction.text import CountVectorizer
```

```
In [5]: from sklearn.model_selection import train_test_split
```

```
In [6]: from sklearn.naive_bayes import MultinomialNB
```

```
In [8]: data['language'].value_counts()
```

```
Out[8]: language
Estonian      1000
Swedish       1000
Thai          1000
Tamil         1000
Dutch         1000
Japanese      1000
Turkish       1000
Latin         1000
Urdu          1000
Indonesian    1000
Portuguese    1000
French        1000
Chinese       1000
Korean        1000
Hindi         1000
Spanish       1000
Pushto        1000
Persian       1000
Romanian      1000
Russian       1000
English       1000
Arabic        1000
Name: count, dtype: int64
```

```
In [9]: data.dtypes
```

```
Out[9]: Text      object
language  object
dtype: object
```

```
In [13]: x = np.array(data['Text'])
y = np.array(data['language'])
```

```
In [14]: cv = CountVectorizer()
X = cv.fit_transform(x)
```

```
In [15]: X_train,X_test,y_train,y_test = train_test_split(X,y, test_size= 0.33, random_st
```

```
In [16]: X_train
```

```
Out[16]: <Compressed Sparse Row sparse matrix of dtype 'int64'
         with 613529 stored elements and shape (14740, 277720)>
```

```
In [17]: print(X_train)
```

```

<Compressed Sparse Row sparse matrix of dtype 'int64'
  with 613529 stored elements and shape (14740, 277720)>
  Coords      Values
(0, 197295)    2
(0, 197708)    1
(0, 197801)    1
(0, 198388)    1
(0, 197467)    1
(0, 197865)    2
(0, 197604)    1
(0, 198428)    1
(0, 198501)    1
(0, 198556)    1
(0, 197332)    1
(0, 197485)    2
(0, 198123)    1
(0, 197892)    1
(0, 197990)    1
(0, 198053)    1
(0, 198417)    1
(0, 197623)    1
(1, 197641)    2
(1, 197314)    1
(1, 197931)    1
(1, 197804)    3
(1, 198397)    1
(1, 197149)    1
(1, 197781)    1
:      :
(14738, 188817)    1
(14738, 192004)    1
(14738, 157171)    1
(14738, 190346)    1
(14738, 190725)    1
(14738, 189685)    1
(14738, 159269)    2
(14738, 145431)    1
(14738, 173292)    1
(14738, 176062)    1
(14738, 159959)    1
(14738, 190198)    1
(14738, 167124)    1
(14738, 168158)    1
(14738, 180260)    2
(14738, 153262)    1
(14738, 162150)    1
(14738, 153355)    1
(14738, 178104)    1
(14738, 163770)    1
(14739, 223002)    1
(14739, 235170)    1
(14739, 222446)    1
(14739, 221922)    1
(14739, 242446)    1

```

In [18]: `y_test`

Out[18]: `array(['Japanese', 'Russian', 'Latin', ..., 'Turkish', 'Arabic',
 'English'], dtype=object)`

```
In [19]: model = MultinomialNB()
```

```
In [20]: model.fit(X_train,y_train)
```

```
Out[20]: ▼ MultinomialNB ⓘ ?
```

```
MultinomialNB()
```

```
In [21]: model.score(X_test,y_test)
```

```
Out[21]: 0.953168044077135
```

```
In [ ]: user = input("Enter a text")  
data = cv.transform([user]).toarray()  
output = model.predict(data)  
print(output)
```

```
In [ ]:
```